Лабораторная работа № 9
Разработка программ с использованием паттернов *Компоновщик, Цепочка обязанностей, Снимок*

Цель работы: научиться применять паттерны проектирования *Компоновщик, Цепочка обязанностей, Снимок* для реализации объекта с дополнительной функциональностью.

1. Теоретические сведения

Теоретические сведения об указанных паттернах, а также примеры их использования можно взять в лекционном курсе.

2. Задания к лабораторной работе

Для заданного варианта задания разработать UML-диаграмму классов и диаграмму последовательности.

В качестве основы взять проект, разработанный для лабораторной работы № 7. При необходимости адаптировать проект для реализации указанных ниже паттернов:

- при помощи паттерна *Компоновщик* сгруппировать множество объектов в *древовидную* структуру;
- при помощи паттерна *Цепочка обязанностей* реализовать передачу запроса на обработку объекта по цепочке обработчиков;
- при помощи паттерна Снимок реализовать сохранение и восстановление состояния объектов.

Разработать консольное приложение (C++, C#). Допускается вводить дополнительные понятия предметной области. В программе предусмотреть тестирование функциональности созданных объектов классов.

Отчет по лабораторной работе – файл формата pdf. Формат имени файла отчета: <НомерГруппы >_<ФамилияСтудента>.pdf. В отчет включить построенные диаграммы и исходный код программы (при необходимости и заголовочные файлы). Формат отчета см. в Приложении. Отчет загрузить в LMS.

При защите лабораторной работы: уметь объяснить логику и детали работы программы; реализацию паттернов проектирования на примере разработанной программы.

Технологии конструирования программного обеспечения

Отчет по лабораторной работе № 00

Группа: 000-000 **Студент**: Иванов Иван Иванович

Задание на лабораторную работу

<Формулировка задания согласно варианту …… ……..>

Диаграмма классов

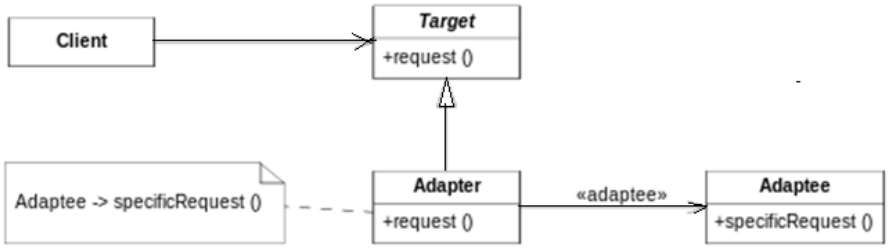
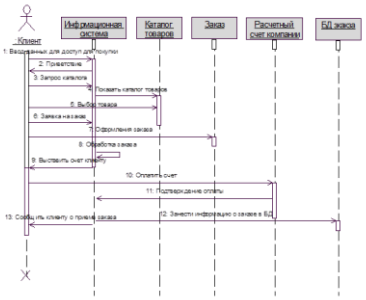


Диаграмма последовательности



Исходный код программы

```
#include "pch.h"
#include "CppUnitTest.h"
#include "../Add/Add.cpp"

using namespace Microsoft::VisualStudio::CppUnitTestFramework;

namespace UnitTest1
{
	TEST_CLASS(UnitTest1)
	{
	public:

		TEST_METHOD(TestMethod1)
		{
			Assert::AreEqual(2, add(1, 1));
		}
```